
Estudo profundo: extrair o conteúdo inteiro de um site com Python

Guia completo + implementação de referência funcionando (sitecrawl/), com 55 testes

Implementação de referência: [sitecrawl/](#) · 55 testes automatizados

Repositório: [studies/web-scraping-python](#)

Gerado em: 12/08/2026

Guia completo + implementação de referência funcionando ([sitecrawl/](#)), com 55 testes automatizados e números medidos, não estimados.

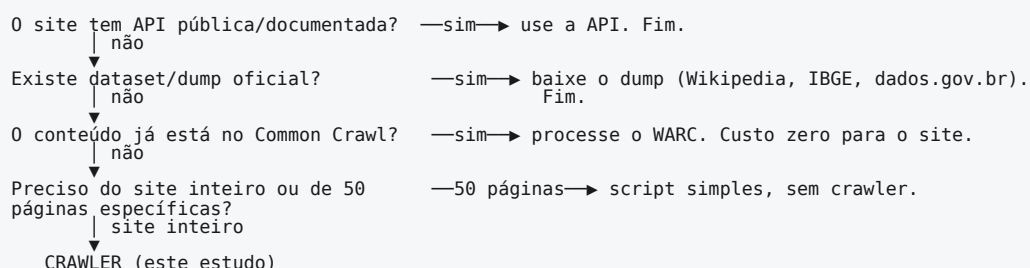
O objetivo aqui não é "aprender BeautifulSoup". É responder à pergunta difícil: **como pegar *todo* o conteúdo de um site, de forma automática, sem quebrar no meio, sem duplicar tudo e sem ser bloqueado?**

Sumário

1. [Decida antes de codar](#)
 2. [O terreno legal e ético](#)
 3. [Anatomia de um crawler completo](#)
 4. [Reconhecimento: mapeie o alvo antes](#)
 5. [Escolha de ferramentas \(com números\)](#)
 6. [Os 10 problemas que realmente quebram um crawler](#)
 7. [Extração: de HTML para conteúdo](#)
 8. [JavaScript: quando o navegador é inevitável](#)
 9. [Anti-bot: o que dá para fazer sem virar vilão](#)
 10. [Escala: de 1 mil a 10 milhões de páginas](#)
 11. [Qualidade do dado](#)
 12. [A implementação de referência](#)
 13. [Checklist final](#)
 14. [Referências](#)
-

1. Decida antes de codar

Raspar um site é a **última** opção, não a primeira. Antes de escrever a primeira linha:



Motivo prático, não moral: uma API devolve dado estruturado e estável. HTML muda de layout sem aviso e quebra seu extrator numa terça-feira à noite. Cada camada acima que você consegue usar elimina uma classe inteira de manutenção.

Common Crawl merece destaque: são petabytes de páginas já rastreadas, mensalmente, com índice por URL. Se o alvo é análise de conteúdo em massa e não precisa ser de hoje, processar WARC é mais barato e não custa nada ao site de origem.

2. O terreno legal e ético

Não sou advogado e isto não é parecer jurídico. Mas ignorar esta seção é o que transforma um projeto técnico em um problema caro.

As quatro camadas independentes

Camada	O que é	Ignorar significa
robots.txt	Padrão técnico (RFC 9309) que declara o que crawlers podem acessar	Não é lei, mas é a prova documental de que você foi avisado
Termos de Uso	Contrato. Muitos proíbem coleta automatizada explicitamente	Quebra contratual; base para ação civil
Direito autoral	O texto do site pertence a alguém	Republicar conteúdo raspado é infração, mesmo com atribuição
LGPD (Lei 13.709/2018)	Dado pessoal tem regras, mesmo público	Coletar nome/e-mail/CPF exposto em site público continua sendo tratamento de dado pessoal e exige base legal

Ponto que quase todo tutorial erra: **"está público na internet" não é base legal sob a LGPD**. O art. 7º §4º trata dado de acesso público, mas o tratamento ainda precisa de finalidade legítima, específica e informada. Raspar perfis públicos para montar uma base de contatos é exatamente o caso que a ANPD já sinalizou como problemático.

Jurisprudência que costumam citar errado

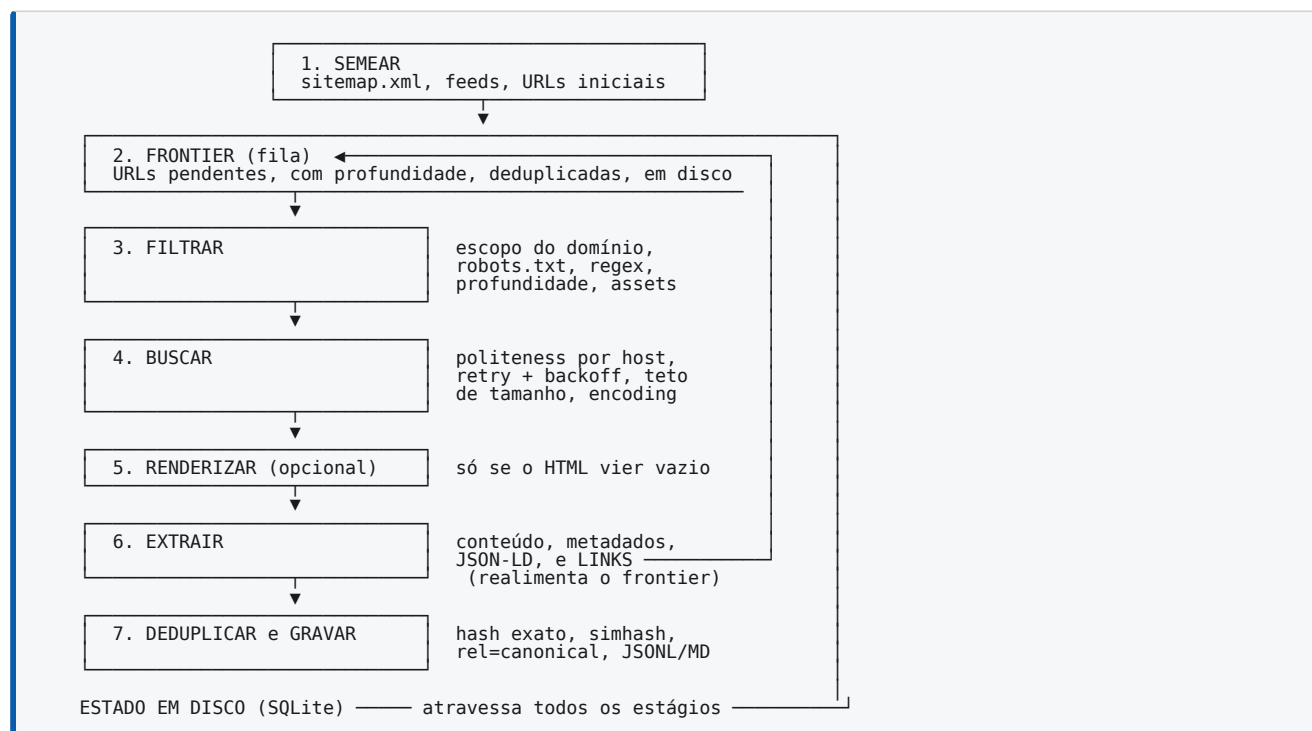
- **hiQ Labs v. LinkedIn** (EUA, 9º Circuito, 2019/2022): decidiu que raspar dado *público* provavelmente não viola o CFAA (lei de acesso não autorizado a computador). Não decidiu que scraping é livre — o caso terminou depois em derrota da hiQ por **quebra de contrato** (os Termos de Uso). A lição real: o CFAA não é o seu risco principal; o contrato é.
- **Van Buren v. United States** (Suprema Corte EUA, 2021): estreitou o CFAA para "acesso a áreas às quais você não tem autorização", não "uso indevido de acesso autorizado". Reforça o mesmo ponto.
- No Brasil, o Marco Civil (Lei 12.965/2014) e o Código Civil dão o enquadramento; sobrecarregar um servidor a ponto de degradar o serviço pode configurar ato ilícito com dever de indenizar, independentemente de scraping.

A regra prática que resolve 95% dos casos

1. Respeite `robots.txt` e `Crawl-delay`. Sempre. Custa uma linha de código.
2. Identifique-se no `User-Agent`, com uma forma de contato. Um admin que vê tráfego estranho e consegue te contatar manda um e-mail; um que não consegue, bloqueia o IP.
3. Uma requisição por segundo por host é educado. Não paralelize contra o mesmo servidor só porque dá.
4. Não colete dado pessoal se a finalidade não exige.
5. Não republique o conteúdo. Extrair para análise $\neq$ redistribuir.
6. Se o site oferece API, use a API mesmo que seja mais chata.

3. Anatomia de um crawler completo

Um crawler sério tem sete estágios. Tutoriais mostram dois (baixar e parsear) e é por isso que o código deles não sobrevive à segunda hora de execução.



O estado em disco é o que separa "script" de "ferramenta". Um crawl de site médio leva horas. Ele **vai** ser interrompido. Sem estado, você recomeça do zero — e bate no servidor com as mesmas 10 mil requisições de novo.

4. Reconhecimento: mapeie o alvo antes

Trinta minutos aqui economizam dias depois. Faça nesta ordem:

4.1 robots.txt — o mapa oficial

```
curl -s https://alvo.com/robots.txt
```

Você quer três coisas: o que é proibido, o `Crawl-delay`, e principalmente as linhas `Sitemap:`. Elas apontam para o índice completo do site.

4.2 sitemap.xml — a lista pronta

Um sitemap pode entregar 50 mil URLs em uma requisição, já com data de modificação. É ordens de grandeza mais barato que descobrir link a link.

```
curl -s https://alvo.com/sitemap.xml | head -40
```

Caminhos comuns quando o robots.txt não declara: `/sitemap.xml`, `/sitemap_index.xml`, `/wp-sitemap.xml` (WordPress 5.5+), `/sitemap.xml.gz`.

Sitemaps se aninham: um `<sitemapindex>` aponta para vários `<urlset>`. Trate a recursão (o código de referência vai até 3 níveis).

4.3 Feeds RSS/Atom

`/feed`, `/rss`, `/atom.xml`, `/index.xml`. Menos completos que sitemap, mas dão o conteúdo recente já limpo — e são a base ideal para coleta incremental.

4.4 A API interna — o atalho que quase ninguém procura

Abra o DevTools (F12) → aba **Network** → filtro **Fetch/XHR** → navegue pelo site.

Se o alvo for uma SPA moderna, você vai ver chamadas do tipo `GET /api/v2/posts?page=3&limit=20` devolvendo JSON puro. Chamar esse endpoint direto:

- é 50-100× mais barato que renderizar a página;
- devolve dado já estruturado, sem seletor frágil;
- geralmente pagina de forma previsível;
- não quebra quando o CSS muda.

Essa descoberta muitas vezes transforma um projeto de duas semanas em um de duas horas.

4.5 Estado embutido no HTML

Mesmo sem API exposta, muitos frameworks serializam o dado dentro da página:

Framework	Onde procurar
Next.js	<code><script id="__NEXT_DATA__"></code>
Nuxt	<code>window.__NUXT__</code>
Redux/genérico	<code>window.__INITIAL_STATE__</code>
Apollo/GraphQL	<code>window.__APOLLO_STATE__</code>
Qualquer um com SEO	<code><script type="application/ld+json"></code>

`sitecrawl/render.py` já procura os quatro primeiros; `sitecrawl/extract.py` sempre captura JSON-LD. JSON-LD é especialmente valioso: é o dado que o site publica *de propósito* para o Google, tipado, com preço, autor, data, avaliação.

4.6 Padrões de paginação

Descubra como o site pagina antes de sair rastejando: `?page=2`, `?offset=20`, `?cursor=abc`, ou scroll infinito (que é sempre XHR — volte ao 4.4).

Armadilha: paginação com filtros combináveis (`?cor=azul&tamanho=M&ordem=preco`) gera explosão combinatória. 5 filtros com 10 valores cada = 100 mil URLs de conteúdo quase idêntico. Bloqueie com `--excluir` desde o início.

5. Escolha de ferramentas (com números)

5.1 Camada HTTP

Biblioteca	Quando usar
<code>requests</code>	Scripts pequenos, síncronos. Simples e onipresente, mas sem async.
<code>httpx</code>	Padrão recomendado. API do requests + <code>async</code> + HTTP/2 + streaming.
<code>aiohttp</code>	Alternativa async madura; API mais verbosa.
<code>curl_cffi</code>	Quando você precisa imitar o fingerprint TLS de um navegador real.

5.2 Parsing de DOM — medido, não achismo

Rodei `benchmark_parsers.py` contra duas páginas reais (x86_64 Linux, Python 3.11.15, mediana de 30 execuções). Tarefa: parsear o documento e extrair todos os `href`.

<https://pypi.org/help/> — **80 KB de HTML**

Operação	Mediana	Relativo
lxml.html puro	1,48 ms	1,0×
selectolax (Lexbor, C)	2,44 ms	1,7×
BeautifulSoup + lxml	22,52 ms	15,2×
BeautifulSoup + html.parser	32,63 ms	22,1×
trafilatura (extração de conteúdo)	37,33 ms	25,3×

<https://pypi.org/classifiers/> — 513 KB de HTML

Operação	Mediana	Relativo
lxml.html puro	7,03 ms	1,0×
selectolax (Lexbor, C)	11,55 ms	1,6×
BeautifulSoup + lxml	87,11 ms	12,4×
BeautifulSoup + html.parser	133,14 ms	19,0×
trafilatura (extração de conteúdo)	405,77 ms	57,8×

Três conclusões que mudam decisões de projeto:

1. **BeautifulSoup custa 12-22× mais que as alternativas em C.** Em 100 mil páginas de 80 KB, isso é 3 min (selectolax) contra 54 min (BS4+html.parser) só de parsing. Para 200 páginas, é irrelevante — use o que for confortável.
2. **lxml puro é o mais rápido**, mas a API do selectolax (seletores CSS diretos, tolerância a HTML quebrado) compensa os ~60% de diferença na maioria dos casos.
3. **A extração de conteúdo é o gargalo real, não o parsing.** O trafilatura sozinho custa mais que todo o resto somado, e escala pior com o tamanho do documento (25× → 58× ao passar de 80 KB para 513 KB). Se precisar otimizar, é aqui — e a saída é `ProcessPoolExecutor`, porque isso é CPU pura, não I/O.

Reproduza no seu alvo: `python benchmark_parsers.py https://seu-site.com/pagina`

5.3 Extração de conteúdo principal

Ferramenta	Abordagem	Observação
<code>trafilatura</code>	Heurística + regras de boilerplate	Melhor equilíbrio; devolve metadados e markdown
<code>readability-lxml</code>	Porte do Readability do Firefox	Mais simples, menos metadados
<code>justext</code>	Classificação de parágrafos por densidade	Bom para corpus linguístico
Seletores CSS próprios	Você escreve as regras	Preciso em 1 site, insustentável em 50

5.4 Frameworks completos

Ferramenta	Use quando
Código próprio (como este)	Você quer entender e controlar cada estágio; até ~100 mil páginas
Scrapy	Produção séria, múltiplos spiders, pipelines, middlewares, +1 M páginas
<code>Playwright</code> / <code>Selenium</code>	Só como camada de renderização, nunca como crawler principal
<code>crawl4ai</code> , <code>Firecrawl</code>	Saída pronta para LLM; menos controle, mais rapidez inicial

Scrapy é excelente e resolve muito do que este estudo implementa à mão. A implementação de referência aqui é didática de propósito: cada mecanismo aparece explícito e comentado, em cerca de 2.000 linhas (mais 700 de teste), em vez de escondido atrás de uma configuração de framework.

6. Os 10 problemas que realmente quebram um crawler

6.1 Explosão de URLs duplicadas

Estas cinco URLs são a mesma página, e um `set()` ingênuo vê cinco:

```
https://ex.com/post
https://ex.com/post/
http://ex.com/post
https://EX.com/post?utm_source=twitter
https://ex.com/post#comentarios
```

Sem canonicalização, o crawler multiplica o trabalho por 5-10× e o servidor te vê como ataque. `sitecrawl/urls.py` normaliza esquema, host, porta, path, parâmetros de rastreamento e fragmento.

Sutileza que quase todo mundo erra: não remova `www.` automaticamente. Muitos sites servem conteúdo diferente entre ápex e `www`, e alguns entram em loop de redirect. Deixe o servidor decidir e deduplique pela URL final.

Outra sutileza: verificar escopo com `host.endswith("ex.com")` aceita `malicioso-ex.com`. Tem que ser `host == "ex.com" or host.endswith(".ex.com")`.

6.2 Armadilhas de crawler (crawler traps)

URLs infinitas que geram conteúdo novo para sempre:

- **Calendários:** `/eventos/2027/03`, `/eventos/2027/04` ... até o fim dos tempos.
- **Busca facetada:** combinações de filtros (\$4.6).
- **Caminhos recursivos:** `/a/b/a/b/a/b/...` por link relativo mal formado.
- **IDs de sessão na URL:** cada visita gera uma URL nova.

Defesas, em ordem de importância: `max_depth`, regex de exclusão, teto de páginas, e detecção de repetição de segmentos no path.

6.3 Encoding

Site brasileiro antigo serve ISO-8859-1 sem declarar no cabeçalho. `response.text` do requests chuta errado e você salva `Ã$Ãfo` no lugar de `ção` — e só descobre depois de 10 mil páginas.

Ordem correta (implementada em `fetch.decode`): charset do cabeçalho HTTP → `<meta charset>` no HTML → UTF-8 → cp1252 → latin-1 com substituição (nunca falha).

6.4 robots.txt: o erro de 5xx

Convenção do RFC 9309 que quase todo tutorial ignora:

Resposta do <code>/robots.txt</code>	Comportamento correto
200 com regras	Obedeça as regras
404 / 410	Libere tudo (não existe robots.txt)
401 / 403	Proíba tudo
5xx ou timeout	Proíba tudo (não presuma permissão)

Tratar 503 como "pode tudo" é o caminho mais curto para um IP banido: o servidor está sobrecarregado e você escolheu justamente esse momento para atacar.

E não use o `urllib.robotparser` da biblioteca padrão. Ele não implementa curingas (`*` e `$`) nos caminhos: trata o `*` como caractere literal e libera tudo que só foi proibido por curinga. Medido contra o robots.txt real do PyPI, que usa `Disallow: /pypi/*/json` e `Disallow: /search*`:

Caminho	urllib.robotparser	protego	O site quis
/simple/requests/	BARRADA	BARRADA	barrar
/pypi/requests/json	PODE	BARRADA	barrar
/search?q=http	PODE	BARRADA	barrar
/pypi?x=1	PODE	BARRADA	barrar

Repare que o erro é para o lado errado: ele **libera** o que foi proibido, silenciosamente. Você acessa o que não devia e nem fica sabendo. `protego` implementa as regras do Google e é uma linha no requirements — `sitecrawl` o usa quando está instalado e emite aviso quando não está.

6.5 Retry, backoff e o `Retry-After`

Três regras:

1. Só repita o que faz sentido: `408, 425, 429, 500, 502, 503, 504`. Um `404` não melhora com insistência.
2. **Obedeça o `Retry-After`**. Um `429` com `Retry-After: 30` é o servidor te dizendo exatamente o que fazer.
3. **Backoff exponencial *com jitter***. Sem jitter, 16 workers que falharam juntos voltam juntos no mesmo milissegundo e derrubam o servidor de novo. $2^{\text{tentativa}} \times (0,5 + \text{aleatório})$.

E um `429` deve deixar o crawler mais lento **permanentemente** naquele host, não só na tentativa seguinte.

6.6 Conteúdo que só existe depois do JavaScript

Sintoma: `curl` devolve `<div id="root"></div>` e nada mais. Ver §8.

6.7 HTTP 200 que não é sucesso

A falha mais traiçoeira do scraping. Cloudflare, Akamai, Fastly e PerimeterX devolvem **status 200** com uma página de desafio.

Isso não é hipótese. Aconteceu enquanto eu testava este código contra o PyPI:

```
6 páginas em 13.9s (0.4 req/s) – 4 salvas, 0 duplicadas, 0 erros,
3 barradas por robots, 2 DESAFIOS ANTI-BOT, 0.7 MB
```

`https://pypi.org/project/requests` respondeu **200 OK** com uma página de 46 palavras intitulada *"Client Challenge"*. Um crawler que só checa `status == 200` teria salvado isso como se fosse a página do pacote — e o erro só apareceria semanas depois.

Detecção (`extract.looks_like_block_page`): pouquíssimo texto e um marcador conhecido (`"just a moment"`, `"client challenge"`, `"/cdn-cgi/challenge-platform"`, `"g-recaptcha"` ...). Exigir os dois sinais evita marcar como bloqueio um artigo que só *fala* sobre captcha.

Primo próximo: o **soft-404** — página "não encontrado" servida com 200. Detecta-se pelo mesmo princípio (texto curto + marcador), ou comparando com a resposta de uma URL aleatória que certamente não existe.

6.8 Duplicação de conteúdo (não de URL)

URLs diferentes, conteúdo igual: e-commerce servindo o produto em várias categorias, versões para impressão, paginação de comentários.

Três camadas, da mais barata para a mais cara:

1. **rel=canonical** — o próprio site declarando qual é a URL oficial. De graça.
2. **SHA-256 do texto normalizado** — pega duplicata exata, sem falso positivo.
3. **SimHash de 64 bits** — pega o *quase*-duplicado (mesma página, bloco de "relacionados" diferente). Distância de Hamming ≤ 3 é o corte clássico.

Sobre o SimHash: comparar cada página nova contra todas as anteriores é $O(n)$ por página — 50 mil páginas viram 1,25 bilhão de comparações. `dedupe.NearDuplicateIndex` divide o hash em 4 bandas de 16 bits e indexa cada banda; pelo princípio da casa dos pombos, dois hashes a distância ≤ 3 obrigatoriamente compartilham ao menos uma banda idêntica, então basta comparar os candidatos daquele balde.

6.9 Estado e retomada

Já dito, mas é o item mais ignorado: grave o frontier em disco. `sitecrawl` usa SQLite com WAL — dá até para inspecionar o progresso com outro processo enquanto o crawl roda:

```
sqlite3 saida/estado.sqlite3 "SELECT outcome, COUNT(*) FROM pages GROUP BY outcome"
```

Detalhe crítico de implementação: ao atingir o limite de páginas, as URLs que sobraram na fila **não podem** ser marcadas como concluídas — senão o `--resume` da próxima execução as considera prontas e o conteúdo se perde para sempre. É um bug silencioso clássico; `test_resume_nao_refaz_trabalho` existe exatamente para travá-lo.

6.10 Memória

Três vazamentos comuns:

- **Frontier na RAM.** 1 milhão de URLs $\times$ ~ 80 bytes ≈ 80 MB só de strings, mais overhead do `set`. Acima de ~ 100 mil URLs, use disco (SQLite) ou um filtro de Bloom.
- **Baixar o que não devia.** Um `.iso` linkado por engano consome toda a RAM do processo. Solução: streaming com corte por tamanho (`Fetcher._stream`), não `client.get()`.
- **Guardar todo o HTML.** Não acumule as páginas numa lista para "processar no fim". Grave incrementalmente (JSONL) e libere.

7. Extração: de HTML para conteúdo

7.1 Separe navegação de conteúdo

São duas tarefas com ferramentas diferentes:

- **Navegação** (links, `<base>`, `rel=canonical`, `meta robots`): parser DOM rápido.
- **Conteúdo** (o artigo, sem menu/rodapé/banner): extrator heurístico.

7.2 A armadilha do `<base href>`

```
<head><base href="https://cdn.exemplo.com/v2/"></head>
```

Essa tag muda a resolução de **todo** href relativo da página. Ignorar isso produz um crawler que gera 404 em massa. Uma linha de código, e quase nenhum tutorial menciona.

7.3 Prefira dado estruturado a seletor

Ordem de preferência para qualquer campo:

1. **JSON-LD** (`<script type="application/ld+json">`) — tipado, estável, publicado de propósito para buscadores.
2. **Meta tags** (`og:title`, `article:published_time`) — quase tão estáveis.
3. **Microdata / RDFa** — menos comum, mesma ideia.
4. **Seletor CSS no corpo** — último recurso, quebra a cada redesign.

7.4 Gotcha real do trafilatura

Descoberto testando: em documentos **curtos**, o trafilatura cai num algoritmo de fallback que **descarta a formatação markdown** e devolve texto corrido — mesmo com `output_format="markdown"`. Num documento de tamanho normal, os mesmos parâmetros devolvem `# Título`, listas e blocos de código corretamente.

Consequência prática: não valide seu pipeline de extração com um HTML de brinquedo de 3 parágrafos. O comportamento que você vai ver não é o de produção.

Por isso `extract._main_content` tem fallback próprio: quando o trafilatura não devolve nada (páginas de índice, contato, listagens), o texto limpo do `<body>` é melhor que campo vazio.

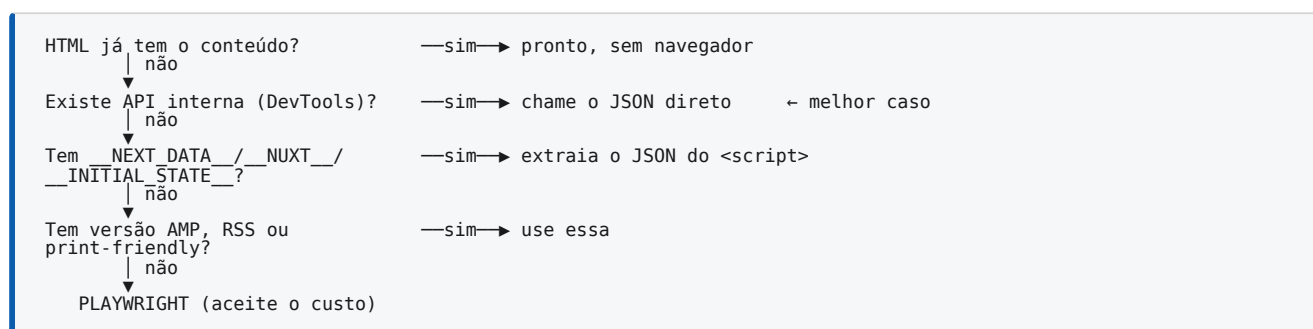
8. JavaScript: quando o navegador é inevitável

8.1 O custo, em ordem de grandeza

Abordagem	RAM por unidade	Tempo por página
Requisição HTTP	~1-5 MB	dezenas de ms
Chromium headless	~100-300 MB por contexto	1-3 s

50-100× mais caro. Renderizar tudo "por garantia" é o erro que transforma um crawl de 20 minutos em um de dois dias.

8.2 A escada de decisão



`sitecrawl` implementa o modo `--render auto`: busca por HTTP primeiro, e só liga o navegador quando a página tem pouco texto e poucos links — a assinatura de uma casca de SPA. Isso mantém o custo do navegador proporcional ao problema real.

8.3 Detalhes que economizam horas

- **Bloqueie imagens, fontes e vídeo** via `context.route()`. Corta cerca de metade do tempo de carregamento. Nunca bloqueie os scripts — são eles que geram o conteúdo.
- **Evite** `wait_until="networkidle"`. Página com polling ou websocket nunca fica ociosa e o wait estoura o timeout. Use `domcontentloaded` + espera curta, ou melhor, `wait_for_selector` no elemento que você realmente quer.
- **Reproveite o `browser_context`**. Subir um Chromium por página é o pior dos mundos.
- **Playwright > Selenium** para scraping novo: API async nativa, interceptação de rede de primeira classe, sem webdriver externo.

9. Anti-bot: o que dá para fazer sem virar vilão

Sinais que os sistemas de proteção observam: taxa de requisição, fingerprint TLS/JA3, ordem dos cabeçalhos HTTP, ausência de `Accept-Language`, User-Agent inconsistente com o resto, padrão de navegação perfeitamente regular, execução (ou não) de JavaScript.

A linha ética e prática:

Legítimo	Território problemático
Identificar-se honestamente no User-Agent	Falsificar UA para se passar pelo Googlebot
Respeitar <code>Crawl-delay</code> e desacelerar em 429	Rotacionar milhares de IPs residenciais para furar limite
Reusar sessão/cookies como um usuário normal	Resolver CAPTCHA em massa via serviço terceirizado
Usar proxy por questão geográfica legítima	Burlar bloqueio explícito ao seu IP
Cachear para não repetir requisição	Ignorar <code>robots.txt</code> "porque dá"

A coluna da direita não é só questionável — é o que transforma "coleta de dados" em "acesso não autorizado" nos termos do contrato do site, e o que gera as ações judiciais.

Se você está sendo bloqueado, a primeira hipótese é que você está rápido demais.

Antes de pensar em proxy: aumente o delay, reduza a concorrência, respeite os 429. Na prática isso resolve a maioria dos bloqueios, porque a maioria dos bloqueios é limite de taxa, não detecção sofisticada.

E se o site bloqueia de forma explícita e persistente, a resposta certa é parar e procurar o canal oficial — muitos sites têm API ou acordo de licenciamento de dados.

10. Escala: de 1 mil a 10 milhões de páginas

Escala	Arquitetura	Frontier	Armazenamento
< 1 mil	Script síncrono	<code>set()</code> na memória	JSON/CSV
1 mil - 100 mil	Async, 1 processo (este código)	SQLite	JSONL + arquivos
100 mil - 1 M	Scrapy, 1 máquina	SQLite/Redis	Parquet, S3
1 M - 10 M	Scrapy distribuído / workers + fila	Redis + filtro de Bloom	S3 + catálogo
> 10 M	Cluster dedicado	Bloom particionado por host	Data lake, formato WARC

Pontos de virada:

- **~100 mil URLs:** o frontier não cabe mais confortavelmente na RAM. Vá para disco.
- **~1 M URLs:** o `set()` de deduplicação vira o gargalo. Filtro de Bloom (falso positivo ~0,1% é aceitável: você perde 1 página em mil e economiza gigabytes).
- **Múltiplas máquinas:** a fila precisa ser particionada **por host**, não por URL, senão duas máquinas batem no mesmo servidor simultaneamente e a politeness some.

Concorrência: como raspagem é I/O-bound, `asyncio` entrega a concorrência de threads sem o custo de contexto. O único gargalo de CPU é o parsing/extração (§5.2) — e a solução é `ProcessPoolExecutor` só para essa etapa, não trocar o modelo de rede.

11. Qualidade do dado

Um crawl que "terminou sem erro" pode ter coletado lixo. Meça:

Métrica	Sinal de alarme
Páginas com <code>word_count < 50</code>	Muitas → extrator falhando ou páginas de desafio
Taxa de duplicatas	> 40% → problema de canonicalização de URL
Distribuição de status HTTP	Muitos 404 → links quebrados ou armadilha
Páginas marcadas <code>blocked</code>	Qualquer número > 0 → desacelere
Cobertura vs. sitemap	Salvou 300 de 5.000 do sitemap? Investigue o filtro
Idioma detectado	Inesperado → veja abaixo

Sobre o idioma: rodando este crawler contra o PyPI, as páginas voltaram em **português** (`lang: pt_BR`) — porque o `Accept-Language: pt-BR` do fetcher acionou a negociação de conteúdo do site. Óbvio depois de descoberto, invisível antes. Se o corpus precisa ser em um idioma específico, **fixe o `Accept-Language` de propósito** e confira o `lang` no resultado.

Validação mínima antes de declarar vitória:

```
# distribuição de resultados
sqlite3 saída/estado.sqlite3 "SELECT outcome, COUNT(*) FROM pages GROUP BY outcome"

# páginas suspeitamente curtas
sqlite3 saída/estado.sqlite3 \
    "SELECT url, word_count FROM pages WHERE outcome='saved' AND word_count < 50 LIMIT 20"
```

E leia 10 páginas extraídas com o olho humano. Nenhuma métrica substitui isso.

12. A implementação de referência

Tudo acima está implementado e testado em `sitecrawl/`.

Instalação

```
cd studies/web-scraping-python
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Opcional, só para sites que dependem de JavaScript:

```
pip install playwright && playwright install chromium
```

Uso

```
# varredura padrão: educada (1 req/s por host), respeita robots, 500 páginas
python -m sitecrawl https://exemplo.com --max-pages 200

# só a seção de blog
python -m sitecrawl https://exemplo.com --incluir '/blog/' --delay 0.5

# site em React que só rende conteúdo depois do JS
python -m sitecrawl https://app.exemplo.com --render auto

# continuar um crawl interrompido (é o padrão – basta o mesmo --saida)
python -m sitecrawl https://exemplo.com --saida ./saida

# ver todas as opções
python -m sitecrawl --help
```

Roteiro de estudo executável

Para *aprender* o mecanismo, rode cada estágio isolado antes de ver o crawler inteiro:

```
python passo_a_passo.py                # alvo padrão: pypi.org
python passo_a_passo.py https://seu-site.com
```

São 10 passos narrados, ~20 requisições ao alvo: robots.txt decidindo URL por URL, sitemap rendendo milhares de URLs numa requisição, canonicalização colapsando variantes, uma busca com anatomia da resposta, extração com metadados e JSON-LD, detecção de página de desafio, gradiente de SimHash, intervalo entre requisições cronometrado, o crawl completo e a inspeção do resultado por SQL.

Como biblioteca:

```
import asyncio
from sitecrawl import CrawlConfig, crawl

config = CrawlConfig(start_urls=["https://exemplo.com"], max_pages=50)
stats = asyncio.run(crawl(config))
print(stats.summary())
```

Saída

```
saída/
├── paginas.jsonl      # uma linha por página: texto, markdown, metadados, JSON-LD
├── markdown/
│   └── exemplo.com/blog/post-1.md # um .md por página, com front matter YAML
├── estado.sqlite3    # frontier + histórico (permite --resume)
└── resumo.json       # estatísticas do crawl
```


Mapa do código

Arquivo	Responsabilidade	Seção do estudo
<code>urls.py</code>	Canonicalização, escopo, nomes de arquivo	§6.1
<code>robots.py</code>	robots.txt com cache e semântica de erro do RFC 9309	§6.4
<code>discovery.py</code>	sitemap.xml (índices, gzip, sem namespace), RSS/Atom	§4.2, §4.3
<code>fetch.py</code>	HTTP async, politeness por host, retry+jitter, encoding	§6.3, §6.5
<code>extract.py</code>	Conteúdo, metadados, JSON-LD, links, detecção de bloqueio	§7, §6.7
<code>dedupe.py</code>	SHA-256, SimHash, índice por bandas	§6.8
<code>render.py</code>	Playwright opcional + estado embutido de framework	§8
<code>store.py</code>	SQLite (estado) + JSONL/Markdown (saída)	§6.9
<code>crawler.py</code>	Orquestração: frontier, workers, limites, estatísticas	§3
<code>cli.py</code>	Interface de linha de comando	—

Testes

```
pip install pytest && python -m pytest tests/ -q
```

```
55 passed in 9.20s
```

Os testes de ponta a ponta sobem um site-fixture em `localhost` (`tests/fixture_site.py`) com `robots.txt`, `sitemap`, `redirect 301`, página duplicada, página proibida, asset CSS e link externo. Testar crawler contra a internet é lento, não determinístico e mal-educado com o alvo.

Bugs reais que o desenvolvimento deste código expôs

Vale registrar, porque são exatamente as classes de defeito que o estudo descreve:

1. **Sondagem custando 121 segundos.** Contra um host inalcançável, o crawler sondava 15 caminhos de `sitemap` × 4 tentativas com `backoff` — antes de o crawl sequer começar. Correção: `retries=0` em requisições especulativas e abortar a sondagem no primeiro erro de rede. Medido: **121 s → 7,1 s**.
2. **Perda silenciosa de URLs no `--resume`.** Ao atingir `max_pages`, as URLs restantes na fila estavam sendo marcadas como concluídas no SQLite — a execução seguinte as pulava e o conteúdo se perdia. Coberto agora por `test_resume_nao_refaz_trabalho`.
3. **Filtro de regex bloqueando a própria semente.** `--incluir '/blog/'` a partir da home fazia o crawl não começar: a URL inicial não casava com o filtro. As sementes agora ignoram `include/exclude` (mas não o escopo de domínio).

Limitações conhecidas

- Não faz login/sessão autenticada.
 - Não extrai texto de PDF (baixa com `--com-documentos`, mas não parseia).
 - Sem `protego` instalado, o crawler cai no parser da `stdlib`, que ignora curingas no `robots.txt` (§6.4). Ele avisa no log, mas o certo é manter o `protego`.
 - Deduplicação e frontier são de processo único; acima de ~100 mil páginas, veja §10.
 - **A demonstração contra site externo neste ambiente foi limitada** pela política de rede do container (só registros de pacote liberados). Os números reais aqui vêm de `pypi.org` (acessível) e do site-fixture local; um crawl de site grande não foi executado.
-

13. Checklist final

Antes de rodar:

- ☐ Confirmei que não existe API oficial nem dump público (§1)
- ☐ Li o `robots.txt` e os Termos de Uso (§2)
- ☐ Meu `User-Agent` me identifica e tem contato (§2)
- ☐ Verifiquei se há sitemap — pode dispensar o crawl de links (§4.2)
- ☐ Abri o DevTools atrás de uma API interna (§4.4)
- ☐ Defini `max_depth`, `max_pages` e regex de exclusão contra armadilhas (§6.2)
- ☐ Delay ≥ 1 s por host, ou o `Crawl-delay` do site se for maior (§2)

Durante:

- ☐ Estado em disco, para poder retomar (§6.9)
- ☐ 429 desacelera o crawler de forma permanente (§6.5)
- ☐ Detecção de página de desafio ligada (§6.7)
- ☐ Deduplicação por canonical + hash + simhash (§6.8)

Depois:

- ☐ Distribuição de `outcome` e de status HTTP conferida (§11)
 - ☐ Cobertura comparada com o total do sitemap (§11)
 - ☐ Li 10 páginas extraídas com olho humano (§11)
 - ☐ Sei qual é minha base legal para o dado coletado (§2)
-

14. Referências

Padrões - [RFC 9309 — Robots Exclusion Protocol](#) - [sitemaps.org](#) — protocolo de sitemap - [schema.org](#) — vocabulário do JSON-LD - [RFC 6585 §4](#) — status 429

Ferramentas - [httpx](#) · [selectolax](#) · [trafilatura](#) · [Scrapy](#) · [Playwright Python](#) · [protego](#)

Fundamentos - Manning, Raghavan & Schütze, *Introduction to Information Retrieval*, cap. 20 (web crawling) - Charikar, *Similarity Estimation Techniques from Rounding Algorithms* (2002) — o SimHash - [Common Crawl](#) — antes de raspar, veja se já está lá

Jurídico (Brasil) - LGPD — [Lei 13.709/2018](#) - [Marco Civil da Internet](#) — [Lei 12.965/2014](#) - [ANPD](#) — orientações e sanções